

Assignment Brief - AI Mock Interview Platform

Objective

Build a full-stack AI-powered mock interview platform where candidates have a real, dynamic voice conversation with an AI interviewer. Not a static question list, not a chat UI. The AI must listen, respond to what the candidate actually said, ask follow-ups, push back, and adapt in real time. The experience should feel like talking to a real interviewer.

Evaluation Criteria

- Does the interview actually feel like a conversation?
 - Clean integration of third-party AI service (cost analysis of this)
 - Code quality and architecture decisions
 - Product thinking and UX judgment
 - Speed of execution and shipping
-

Core Experience

When a candidate lands on the platform:

1. They sign up and set up a brief profile (name, job role, experience level)
2. They choose what kind of interview they want to practice
3. The AI interviewer starts the voice session, introduces itself, and kicks off naturally
4. The conversation is fully dynamic:
 - The AI listens to what the candidate says and responds to that specific answer
 - It asks follow-up questions ("Can you walk me through that more?", "What would you have done differently?")
 - It probes weak answers and acknowledges strong ones
 - It does not move on a fixed script

5. (The candidate can end the session at any point
6. (A detailed feedback report is generated from the full conversation

Interview Types (Candidate Picks at Start)

Type	What it tests
Behavioral	Communication, STAR structure, self-awareness
Technical	Depth of knowledge, problem-solving approach
System Design	Architecture thinking, tradeoffs, communicating complexity
HR / Culture Fit	Motivation, values, situational judgment

The AI interviewer persona and question strategy should adapt based on the type selected.

The Interaction Requirement ~ Most Important

This is the hardest and most important part of the assignment.

The AI must not:

- (Show 10 questions upfront and wait for answers
- (Move to the next question regardless of what was said
- (Give generic responses that could apply to any answer

The AI must:

- (Start with an opening (role, what the interview will cover)
 - (Ask one question at a time
 - (Actually process the candidate's response before deciding what comes next
 - (Follow up when an answer is vague, incomplete, or interesting
 - (Know when to move on vs. when to dig deeper
 - (Close the interview naturally when time is up or coverage is sufficient
-

Technical Scope

Stack

- { **Frontend**: React (Vite or Next.js)
- { **Backend**: Node.js (Express or Fastify) or Next.js API routes
- { **Database**: PostgreSQL
- { **Auth**: Simple JWT. No OAuth.

Voice Layer

The interview must be voice only. No text chat UI. Use a managed voice AI service, do not build or host your own models. The service must support real-time voice conversation with a programmable AI agent.

You are free to use any third-party managed service or any open source solution running locally. No preference on which one. No registration required on our end.

- { Do not hardcode a question bank
- { The AI's next response must always be generated from the full conversation context so far
- { A good implementation maintains a running conversation history per session so the AI always knows what was said

Auth & Accounts

- { Candidates sign up themselves, no pre-filled accounts
- { Simple email + password, JWT issued on login
- { No OAuth, no magic links

What We're Not Looking For

- { A quiz app with a list of preset questions
- { A chatbot that gives the same response regardless of what the user said

- { Training or hosting your own models
 - { Feature breadth over depth, one interview type done properly beats four done poorly
-

Deliverable

A working full-stack app with:

- { Sign up, choose interview type, conduct a real dynamic voice interview, receive feedback report
 - { Dashboard with past sessions and reports
 - { **README.md** with local setup in under 5 commands
 - { Short Loom walking through core loop working end-to-end
-

Bonus Tip: Consider LangGraph for the Conversation Engine

If you want to go beyond a basic LLM chat loop, LangGraph is worth exploring. It's not required, but if you can model the interview as a graph and make it work intuitively, that will stand out.

Why it fits this problem well:

- { **Adaptive branching**: If a candidate gives a weak answer, route to a follow-up node. If they do well, increase difficulty. This branching logic is exactly what graphs are built for.
- { **Persistent state**: A graph node can carry the full session state, interview stage, transcript, running scores, resume context, job description, previous answers, and timing, across the entire conversation.
- { **Specialized nodes**: One node for question generation, one for answer evaluation, one for difficulty adjustment, one for final feedback and scoring. Doing all of this in one monolithic prompt is fragile.
- { **Future-proofing**: Features like pause/resume, human reviewer intervention, coding rounds, or swapping in a different interviewer persona become natural extensions of the graph.

This is a bonus. Nail the core interactive voice loop first.

What Good Looks Like

A strong submission will feel genuinely interactive. Someone running through a behavioral interview should come away feeling like they actually practiced, not like they answered a Google Form. The AI should surprise them at least once: a follow-up they didn't expect, a challenge to something they glossed over, a moment where it clearly understood what they said.

That is the bar.